Ryder Pongracic

Atlanta, GA

678-543-4209 | ryderjmp@gmail.com | linkedin.com/in/rpongracic

github.com/ryderpongracic1 | ryderpongracic1.github.io

Education

Georgia Institute of Technology — **Jan. 2027 — May 2028**

Master of Science in Computer Science | Specialization: Artificial Intelligence — *Atlanta, GA*

Georgia Institute of Technology — **Jun. 2023 — Dec. 2026**

B.S. Computer Science, GPA 3.85/4.0 | Concentration: Intelligence & People — *Atlanta, GA*

Experience

Amazon — **Jun. 2026 — Present**

Software Development Engineer Intern | FinTech Org — *Arlington, VA*

- Authored the approved design doc & shipped a **Step Functions-orchestrated** asset reclassification service end-to-end across **80 production CRs** (Java Lambda, **Smithy** APIs, TypeScript **CDK**) at a **100% line/branch coverage** gate, cutting a **2–3 week** manual cross-team process to **under 5 minutes** & closing a data-integrity gap that silently orphaned assets.
- Built the execution layer on a **Distributed Map** (25-asset batches, 25× concurrency) over a batch subledger API with **DynamoDB TransactWriteItems** sentinel locking & deterministic idempotency keys for **exactly-once** execution; **reclassified 13,368 assets in a 60-second run across 535 batches, 0 failures**, scaling **sublinearly** (+36% volume, +24% wall clock) at a measured **$0.03 per 10K assets** of infrastructure cost.
- Replaced a paginated API's **10K-record hard ceiling** with an **Athena UNLOAD** enumeration adapter, streaming manifests for **5.77M+ records** across all 225 categories to S3 with parameterized queries & failsafe retry polling.
- Root-caused an **org-wide pipeline outage** outside my project: a hardcoded canary assertion tripped a CodeDeploy alarm halting all Lambda traffic shifts behind an opaque CloudFormation error; shipped the fix & restored deploys.

Georgia Institute of Technology — **Jan. 2024 — Dec. 2025**

Computational Researcher | Bhamla Lab & School of Civil Eng. — *Atlanta, GA*

- Built a **MATLAB stochastic simulator** (**10K+ Gillespie trajectories**) & **LangGraph** multi-agent financial model, identifying a **15% copy-trader ratio** as the optimal control threshold.
- Engineered a custom **High Boost filter** & **image interpolation** pipeline across a **2,000-image 3D laser dataset**, improving avg max **IoS/IoP 0.69 → 0.95** & salvaging **401** failed homographies.

Projects

distrikv – Distributed Key-Value Store | *Go, gRPC, LSM-Trees, Raft, Docker* — **Apr. 2026 — Aug. 2026**

- Architected a **Golang** distributed key-value store as a strictly consistent **CP System** where a **Consistent Hash Ring** owns data replication & **Raft consensus** carries only cluster-wide node health — no dedicated liveness probes; serves **6K reads/s** at **p99 1.7 ms** & **1.2K writes/s** at p99 4.6 ms on a 3-node cluster under open-loop Poisson load.
- Engineered an **LSM-Tree Storage Engine** with **leveled compaction**, **write-stall backpressure**, a **32-shard LRU block cache** (**97.2%** Zipfian vs. **70.9%** uniform hit rate across 500K keys), & restart-safe write ordering via a **42-bit incarnation epoch + 22-bit counter** packed into the existing uint64 — zero wire-format changes.
- Implemented **Raft log replication** (crash-persistent log, apply-on-commit) & verified **linearizable histories** under kill-restart & forced leader-election faults with a **Porcupine harness** & **Jepsen-style chaos runner**; anti-entropy replayed ∼**104K refused-but-applied writes** as **373 deduplicated catch-up entries**, converging in **1–5 s**.

vexq – Parallel Vectorized Analytical Query Engine | *Go, C++/AVX2, SQL* — **May 2026 — Aug. 2026**

- Engineered a complete **Golang** analytical query pipeline: custom **columnar format** with per-row-group **zone maps**, dictionary-encoded strings (equality → integer compare), & **CRC32** block integrity; correctness held by a **72-query golden oracle** run across **4 execution paths** (serial, parallel, optimizer-off, stacked-filter) under `-race` in CI.
- Built a **vectorized Volcano-model execution engine** with 1024-row **L1-cached batches**, **selection-vector filters**, a **Pratt SQL parser**, & predicate-pushdown/join-column-pruning optimizer; **within 1.16× of DuckDB end-to-end on TPC-H Q1 (22 ms vs. 19 ms steady-state** all cores, same machine), within **1.2×** single-threaded; isolated **C++/AVX2** kernel study measured the SIMD filter ceiling at **3.3×**.
- Diagnosed & removed the parallel scaling limiter: **GOGC ablation** isolated Go GC at **26–50%** of parallel wall time, then an allocation campaign (**flat open-addressed join table**, per-worker pipeline reuse, scratch-buffer pooling) cut build allocations **1.2M → 1.5K/op** & GC's share to **0–19%**, lifting Q1 to **8.6×** scaling on 14 cores.

Technical Skills

Languages: Python, C/C++, Go, Java, TypeScript/JavaScript, SQL, Cypher, MATLAB

Frameworks & Tools: AWS (Lambda, Step Functions, DynamoDB, S3, Athena, CDK), gRPC, React, FastAPI, OpenMP, Docker, PostgreSQL, Neo4j, GitHub Actions, Linux/Bash

Concepts: Distributed Systems, LSM-Trees, Raft, Vectorized Execution, Query Optimization, Parallel Processing, CI/CD